

Batch Size and Gradient Accumulation

How many examples per step, and what if they don't fit · video 2:28

This lesson answers two questions: how big should a training batch be, and what can you do when it doesn't fit in memory? Bigger batches give gradients closer to the true one (cosine similarity 0.17 for one sequence, 0.90 for 256), but for the same data they mean fewer steps: here, batches of 8 reached a loss of 1.660 and batches of 128 only 1.930. Big batches win on hardware speed.

Gradient accumulation gives the gradient of a big batch with the memory of a small one.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 5).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Noisy gradients	▶ 0:08-0:41	2	☐
2. Same data, different batch sizes	▶ 0:41-1:44	3	☐
3. Gradient accumulation	▶ 1:44-2:18	2	☐

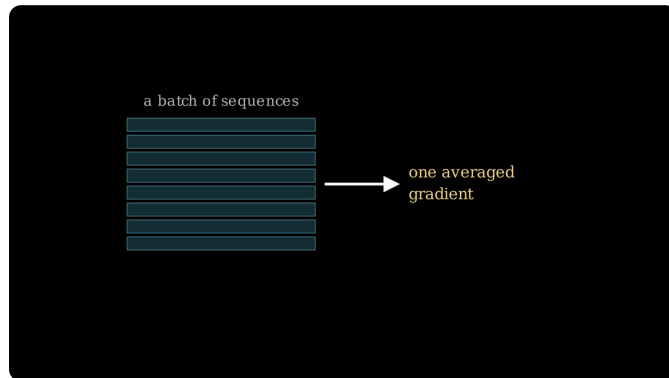
Before you start: Deep Dive episodes 18 (backprop) and 19 (AdamW). Code: code/d21_batch_size (PyTorch only; about 15 minutes on a CPU).

CONCEPT 1 OF 3

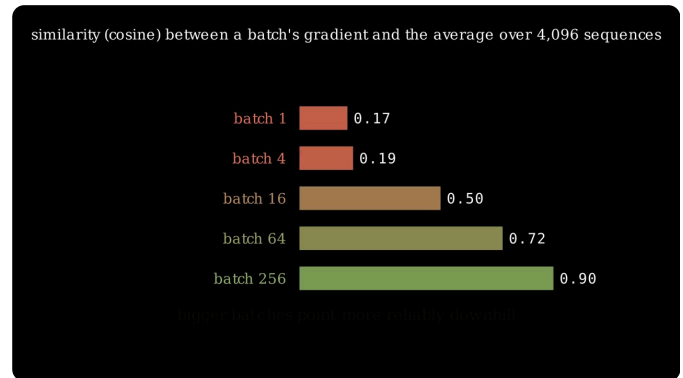
Noisy gradients

▶ Video 0:08–0:41

Every training step averages the gradient over a **batch** of examples. A small batch gives a noisy estimate of the true gradient. After 300 training steps, each batch's gradient is compared with the average over 4,096 sequences (cosine similarity): batch 1 **0.17**, 4 0.19, 16 **0.50**, 64 **0.72**, 256 **0.90**. Bigger batches point more reliably downhill.



▶ 0:15 Each step averages the gradient over a batch of sequences.



▶ 0:39 Similarity to the gradient of 4,096 sequences: 0.17 (batch 1), 0.50 (16), 0.72 (64), 0.90 (256).

KEY IDEA

The batch gradient is an estimate of the full-data gradient; its noise shrinks as the batch grows.

Check 1: Noisy gradients

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 TRUE OR FALSE

"A batch of 256 gives exactly the true gradient."

True ☐ False ☐

1.2 SHORT ANSWER

Why is a single sequence's gradient so different from the average (similarity 0.17)?

.....

CONCEPT 2 OF 3

Same data, different batch sizes

▶ Video 0:41-1:44

A bigger batch means fewer steps for the same data. With 48,000 training sequences in every run: batch 8, 6,000 steps, loss **1.660**; batch 32, 1,500 steps, **1.709**; batch 128, 375 steps, **1.930**. Doubling the learning rate for batch 128 helps (**1.811**) but it still trails. For a fixed amount of data, more, noisier steps won here.

So why use big batches? **Speed**: on this CPU a batch of 8 costs 2.82 ms per sequence, a batch of 32 only 1.56 ms, because the hardware does more work in parallel. Here it was already saturated at 32 (1.68 ms at 128); GPUs keep gaining up to much larger batches. Small batches use the data best; big batches use the hardware best. Large language models train with batches of millions of tokens, with learning rates tuned to match.

48,000 training sequences in every run

batch	learning rate	steps	val loss
8	0.001	6,000	1.660
32	0.001	1,500	1.709
128	0.001	375	1.930
128	0.002	375	1.811

fixed data: more, noisier steps won here

time per training sequence, this CPU



saturated at 32 here; GPUs keep gaining up to much larger batches

▶ 1:10 48,000 sequences: batch 8 → 1.660, 32 → 1.709, 128 → 1.930 (1.811 with twice the learning rate).

▶ 1:30 Time per sequence on this CPU: 2.82 ms (batch 8), 1.56 ms (32), 1.68 ms (128).

KEY IDEA

Batch size trades data efficiency (more steps) against hardware efficiency (more parallel work per step).

Check 2: Same data, different batch sizes

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

With 48,000 sequences and batch 64, how many steps?

.....

2.2 CALCULATION

How long would the 48,000 sequences take at batch 8 and at batch 32, at the measured time per sequence?

.....

2.3 MULTIPLE CHOICE

At a fixed amount of data, which run reached the lowest loss?

- A. Batch 128, lr 0.002
- B. Batch 32
- C. Batch 8
- D. Batch 128, lr 0.001

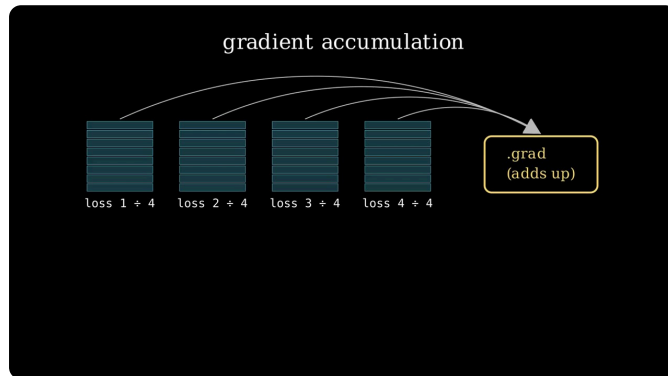
CONCEPT 3 OF 3

Gradient accumulation

▶ Video 1:44-2:18

When the batch you want doesn't fit in memory, use **gradient accumulation**: run several small micro-batches, divide each loss by their number, and call `backward()` each time without clearing the gradients: PyTorch adds them up in `.grad`. Then take one optimizer step.

Four micro-batches of 8 give the same gradient as one batch of 32 to within 1.5×10^{-8} (gradient size 1.123). The memory of a batch of 8, the gradient of 32; just no speed-up, since the micro-batches run one after another.



4 micro-batches of 8 vs one batch of 32:
largest gradient difference 1.5×10^{-8}

the memory of 8, the gradient of 32
(but no speed-up)

▶ 1:57 Run micro-batches, divide each loss by their number, call backward without clearing: the gradients add up.

▶ 2:09 4 micro-batches of 8 = one batch of 32, to 1.5×10^{-8} : the memory of 8, the gradient of 32.

KEY IDEA

Gradients are sums, so they can be accumulated across micro-batches; dividing each loss by the number of micro-batches turns the sum into the average.

Check 3: Gradient accumulation

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 SHORT ANSWER

What goes wrong if you forget to divide each micro-batch loss by 4?

3.2 HANDS-ON CODE

Try it yourself. Accumulate 8 micro-batches of 4 instead, and compare with the batch of 32.

```
model.zero_grad()
for k in range(8):
    (lm_loss(model, x[4*k:4*k+4], y[4*k:4*k+4]) / 8).backward()
print((big - flat_grad(model)).abs().max())
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Noisy gradients · rewatch ▶ 0:08-0:41

1.1 False. Its similarity to the 4,096-sequence gradient is 0.90, not 1: still an estimate.

1.2 It reflects the quirks of that one piece of text; averaging over many sequences cancels those and keeps what they share. Noise in an average shrinks as more independent samples are added.

2. Same data, different batch sizes · rewatch ▶ 0:41-1:44

2.1 750. $48,000 / 64$.

2.2 About 135 s and 75 s. $48,000 \times 2.82 \text{ ms}$ and $48,000 \times 1.56 \text{ ms}$.

2.3 C. 1.660 with 6,000 steps.

3. Gradient accumulation · rewatch ▶ 1:44-2:18

3.1 The accumulated gradient is 4 times too large, as if the learning rate were 4 times higher (for SGD; Adam is mostly insensitive to this scale). Each backward adds a full-size gradient; the sum of 4 is $4 \times$ the average.

3.2 About 10^{-8} : the same gradient again. Any split of the batch gives the same average.